

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 1

Concurrency in Practice

Disciplina: Programarea în Rețea

A efectuat:
student gr. FAF-243
Daniel Nicorici

A verificat:
Cristofor Fistic

Chișinău — 2026

Cuprins

Purpose of the work **2**

1 Source code **2**

2 Hardware and software **2**

3 Language: Python without the GIL **2**

4 Data **3**

5 The program **3**

5.1 Parsing and counting . . . 3
5.2 Task 1a: dividing the file without losing a line . . . 4
5.3 Task 1b: three ways to count . . . 5
5.4 Task 1c: producer and consumers . . . 5
5.5 Task 2: many files . . . 6

6 Task 1 results **6**

6.1 Task 1a . . . 6
6.2 Task 1b . . . 7
6.3 Task 1c . . . 8
6.4 Answers to the questions of Task 1 . . . 8

7 Task 2 results **10**

8 Conclusion **11**

Purpose of the work

The purpose of this lab is to measure how the number of threads, and the way the work is divided among them, change the running time and the memory of a real workload: counting the negative, positive and zero numbers in a file of 50 000 000 integers. From my own measurements I decide when the work is CPU-bound and when it is I/O-bound, I observe a race condition and the cost of fixing it, and I find where the parallel speed-up stops.

1 Source code

The source code is in the Git repository `https://github.com/Nikorici/PR`, folder `concurrency-lab1`. Its `README.md` has the instructions to build and run every task.

2 Hardware and software

Item	Value
Machine	ASUS ROG Zephyrus G14 (GA403UM) laptop
Power	on battery, power profile “balanced”
CPU	AMD Ryzen 9 270 (Zen 4), up to 5.26 GHz, L3 cache 16 MiB
Physical cores	8 (1 socket)
Logical processors	16 (2 per core, SMT)
RAM	32 GB (30 GiB usable)
Disk	Micron MTFDKBA1T0QGN, 1 TB NVMe SSD, ext4
USB drive	Generic Flash Disk, 16 GB (14.8 GB usable), Alcor Micro 058f:6387
	USB 2.0 (480 Mbit/s in `lsusb -t`), FAT32
OS	Arch Linux, kernel 7.2.11-arch1-1
Language	Python 3.14.8, free-threading build (`python3.14t`, GIL disabled)

Table 2.1 – Hardware and software used for every measurement

3 Language: Python without the GIL

The lab allows any language that provides OS threads. Python’s `threading.Thread` is a real OS thread, but the standard CPython interpreter has a *global interpreter lock* (GIL): only one thread at a time may run Python code. With the GIL, threads help when they wait (for a disk, for the network), but they cannot make a CPU-bound job faster.

Since Python 3.13 CPython also ships a *free-threaded* build without the GIL (PEP 703), and in Python 3.14 this build is officially supported (PEP 779). Its executable is `python3.14t`. In it, threads run Python code on different cores at the same time, so it is a fair language for this lab. Every result in this report was measured with `python3.14t`.

To show the difference, I ran Task 1a once more with the GIL turned back on (`python3.14t -X gil=1`, same program, same file). The orange line in Figure 6.1 is that run: with the GIL, 16 threads take 4.10 s instead of 3.46 s on one thread, so more threads only make it slower.

4 Data

`gen.py` writes `data/numbers.txt`: 50 000 000 integers from $-1\,000\,000$ to $1\,000\,000$, one per line (`random.Random` with the fixed seed 20261022, so the file can be generated again identically). The file has 369 438 541 bytes. In the same pass it writes the 8 parts `data/parts/numbers_1.txt` ... `numbers_8.txt` of 6 250 000 numbers each, in order (about 46.2 MB each); `cmp` confirmed that the 8 parts, concatenated, are exactly `numbers.txt`.

The reference totals are counted by the generator from the integers themselves, not by reading the file back, so they are an independent check:

$$\text{negative} = 24\,995\,180, \quad \text{positive} = 25\,004\,796, \quad \text{zero} = 24, \quad \text{total} = 50\,000\,000.$$

They are stored in `data/expected.txt`. The script `run.sh` compares every single run with them and stops at the first wrong total (except Task 1b version 1, whose wrong totals are the point).

5 The program

One program, `lab.py`, does every task; the task and its settings are command-line arguments:

```
python3.14t lab.py 1a <file> <threads>
python3.14t lab.py 1b <file> <version 1|2|3> [threads]
python3.14t lab.py 1c <file> <workers>
python3.14t lab.py 2  <folder> <threads>
```

Each run prints one CSV line `task,threads,time_ms,peak_kb,negative,positive,zero,total`. The time is measured with `time.perf_counter()` around the counting only; the peak memory is the maximum resident set size of the process, from `getrusage()`.

5.1 Parsing and counting

Every task uses the same two helpers. `count` counts the signs of a sequence of integers. `blocks` reads a stream in blocks of at most 64 KiB and cuts each block after its last newline; the incomplete line at the end is carried over to the next block, so a number is never cut in two. Parsing is `int()` on every line.

```
def count(nums):
    neg = pos = zero = 0
    for n in nums:
        if n < 0:
            neg += 1
        elif n > 0:
            pos += 1
        else:
            zero += 1
    return neg, pos, zero

def blocks(f, size):
    carry = b""
    while size > 0:
        chunk = f.read(min(BLOCK, size))
        if not chunk:
            break
```

```
        size -= len(chunk)
        buf = carry + chunk
        cut = buf.rfind(b"\n") + 1
        carry = buf[cut:]
        yield buf[:cut]
    yield carry

def count_stream(f, size):
    return total(count(map(int, b.split())) for b in blocks(f, size))
```

Listing 5.1 – Counting and block reading (lab.py)

The block size matters for memory. My first version used 4 MiB blocks: 16 threads then needed 1.33 GB and 0.88 s, because every block turns into more than half a million Python objects at once. With 64 KiB blocks the same run needs 156 MB and 0.57 s.

5.2 Task 1a: dividing the file without losing a line

With T threads, thread i gets the byte range from $\lfloor i \cdot size/T \rfloor$ to $\lfloor (i+1) \cdot size/T \rfloor$. These byte boundaries usually fall in the middle of a line, so the rule is: **a line belongs to the part that contains its first byte**.

```
def line_start(f, pos):
    if pos == 0:
        return 0
    f.seek(pos - 1)
    f.readline()
    return f.tell()

def task_1a(path, threads):
    size = os.path.getsize(path)
    results = [None] * threads

    def work(i):
        with open(path, "rb") as f:
            start = line_start(f, size * i // threads)
            end = line_start(f, size * (i + 1) // threads)
            f.seek(start)
            results[i] = count_stream(f, end - start)

    run_threads(work, threads)
    return total(results)
```

Listing 5.2 – Task 1a (lab.py)

`line_start(f, pos)` returns the start of the first line that begins at or after `pos`. It reads from `pos - 1` to the end of that line: if the byte before `pos` is already a newline, only that newline is consumed and the line starting exactly at `pos` is kept. Each thread opens the file itself, reads from `line_start(lo)` to `line_start(hi)`, and the next thread starts exactly at that same `line_start(hi)`. So the parts touch without a gap and without overlap: no line is skipped and none is counted twice. No thread reads the whole file; the main thread only starts the threads and adds their three numbers.

`test_lab.py` checks this on small files with 1, 2, 3, 4, 7, 16 and 64 threads (more threads than lines, a file without a final newline, blocks of 7 bytes so numbers are cut by the block reader), and the 50-million totals match in every run.

5.3 Task 1b: three ways to count

The numbers are first loaded into an `array('i')` (not measured). The array is split into 16 equal slices, one per logical processor. Versions 1 and 2 differ in a single line, the lock:

```
counts = [0, 0, 0]
lock = threading.Lock()

def unsynchronised(i):
    for n in part(i):
        k = 0 if n < 0 else 1 if n > 0 else 2
        counts[k] += 1

def locked(i):
    for n in part(i):
        k = 0 if n < 0 else 1 if n > 0 else 2
        with lock:
            counts[k] += 1

def per_thread(i):
    results[i] = count(part(i))
```

Listing 5.3 – Task 1b (lab.py)

5.4 Task 1c: producer and consumers

The queue is Python's `queue.Queue(maxsize=100)`: a list protected by a lock, with two condition variables, so `put` waits while the queue is full and `get` waits while it is empty. The reader closes it with `Queue.shutdown()` (new in Python 3.13): after that `get` still returns the batches left in the queue, and raises `ShutDown` once it is empty, which is exactly when a worker must stop. The `finally` closes the queue even if reading fails, so workers can never wait forever.

```
q = queue.Queue(maxsize=QUEUE_BATCHES)          # 100 batches

def reader():
    try:
        with open(path, "rb") as f:
            batch = []
            for line in f:
                batch.append(line)
                if len(batch) == BATCH:         # 10 000 lines
                    q.put(batch)
                    batch = []
        if batch:
            q.put(batch)
    finally:
        q.shutdown()

def worker(i):
    parts = []
    while True:
        try:
            batch = q.get()
        except queue.ShutDown:
            break
        parts.append(count(map(int, batch)))
```

```
    results[i] = total(parts)
```

Listing 5.4 – Task 1c (lab.py)

5.5 Task 2: many files

A file is the unit of work. The 8 files are dealt to the threads round-robin (`files[i::threads]`), which gives 8, 4 and 2 files per thread for 1, 2 and 4 threads. A thread reads each file with a single `f.read()` into one `bytes` buffer, then parses that buffer with the same block helper (wrapping it in `io.BytesIO`, which does not copy it).

```
def work(i):
    parts = []
    for path in files[i::threads]:
        with open(path, "rb") as f:
            buf = f.read()
        parts.append(count_stream(io.BytesIO(buf), len(buf)))
    results[i] = total(parts)
```

Listing 5.5 – Task 2 (lab.py)

6 Task 1 results

Every configuration was run 3 times; the tables and charts show the median. All the totals were correct, except Task 1b version 1. The file is read from the page cache: after the first run it is in RAM, so these runs measure the CPU, not the SSD.

6.1 Task 1a

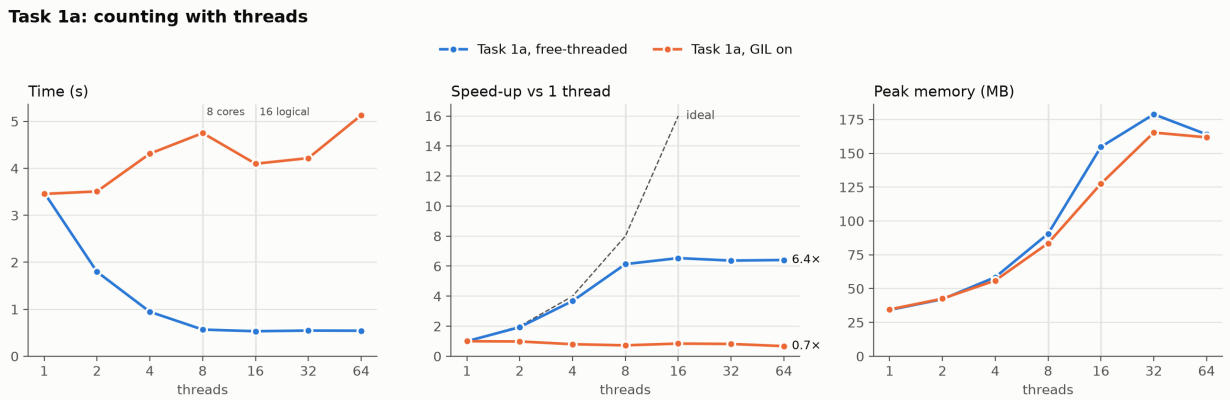


Figure 6.1 – Task 1a: time, speed-up and peak memory against the number of threads

	free-threaded			GIL on (1 run)	
threads	time (s)	speed-up	peak memory (MB)	time (s)	speed-up
1	3.483	1.00	34.2	3.457	1.00
2	1.798	1.94	42.2	3.509	0.99
4	0.947	3.68	58.5	4.313	0.80
8	0.568	6.14	90.6	4.752	0.73
16	0.533	6.54	154.7	4.099	0.84
32	0.547	6.37	178.8	4.218	0.82
64	0.543	6.41	164.0	5.130	0.67

Table 6.1 – Task 1a, medians (`results/1a.csv`, `results/1a-gil.csv`)

6.2 Task 1b

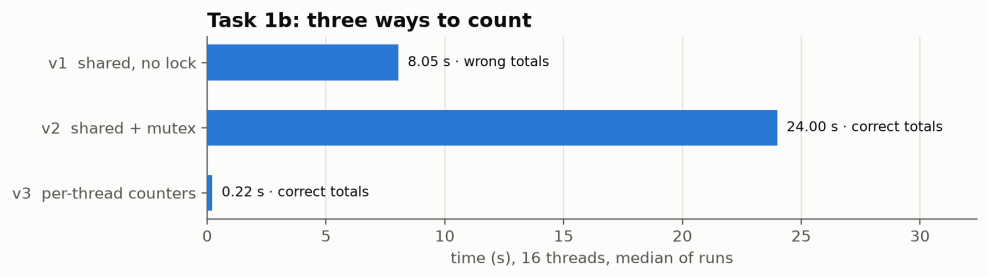


Figure 6.2 – Task 1b: the three versions with 16 threads

version	run	time (s)	negative	positive	zero	total
v1 shared, no lock	1	8.051	5 147 602	4 979 625	24	10 127 251
	2	8.944	5 080 711	4 976 961	24	10 057 696
	3	7.691	5 044 975	4 790 268	24	9 835 267
v2 shared + mutex	1	22.629	24 995 180	25 004 796	24	50 000 000
	2	25.491	24 995 180	25 004 796	24	50 000 000
	3	23.998	24 995 180	25 004 796	24	50 000 000
v3 per-thread counters	1	0.256	24 995 180	25 004 796	24	50 000 000
	2	0.197	24 995 180	25 004 796	24	50 000 000
	3	0.218	24 995 180	25 004 796	24	50 000 000

Table 6.2 – Task 1b, every run, 16 threads (`results/1b.csv`)

Task 1b runs only with 16 threads, as many as the logical processors, as the task requires, so its chart compares the three versions instead of thread counts. The peak memory was 587–618 MB in every run of all three versions. It is set by loading the 50 million numbers before the counting starts: loading alone reaches 593 MB, of which the array itself is 190 MB. So it does not depend on the version.

6.3 Task 1c

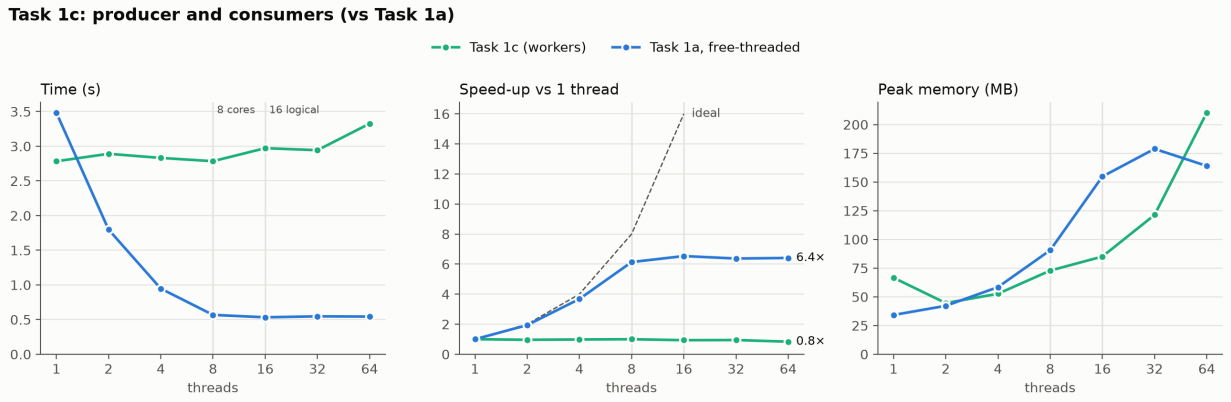


Figure 6.3 – Task 1c compared with Task 1a; the x axis is the number of workers for 1c (plus one reader)

workers	time (s)	speed-up	peak memory (MB)
1	2.783	1.00	66.6
2	2.891	0.96	44.6
4	2.831	0.98	52.7
8	2.784	1.00	72.8
16	2.972	0.94	85.1
32	2.943	0.95	121.4
64	3.326	0.84	210.4

Table 6.3 – Task 1c, medians (`results/1c.csv`)

6.4 Answers to the questions of Task 1

1. At which number of threads does the speed-up stop growing? At 8 threads, the number of physical cores. Up to 8 it is close to linear: 1.94× on 2 threads, 3.68× on 4, 6.14× on 8, because every thread gets a core of its own. From 8 to 16 it only goes from 6.14× to 6.54× (+6%): threads 9–16 run on the second logical processor of cores that are already busy. Two logical processors of one core share its execution units and caches, and the interpreter loop already keeps one core busy, so the second one adds little. After 16 there is no free processor left, and the speed-up stays at 6.4×. It is 6.1× and not 8× on 8 threads mainly because a busy CPU runs all its cores at a lower clock than the 5.26 GHz one core can reach alone (on battery, too), and the threads share the memory bus and the L3 cache.

2. Why do 32 and 64 threads not help? Are they slower? There are only 16 logical processors, so at most 16 threads run at any moment; the others wait for the scheduler. The work is the same, it is only cut into smaller pieces, and every extra thread costs a creation, a stack and context switches. Measured: 0.547 s and 0.543 s against 0.533 s on 16 threads, 2–3% slower, which is within the run-to-run spread (the three runs on 8 threads range from 0.527 to 0.822 s). So they are not faster, and at most slightly slower. With the GIL on the effect is much clearer: 64 threads take 5.13 s against 3.46 s on one, because all of them compete for the same lock.

3. Does the peak memory depend on the number of threads? Why? Yes: 34 MB on 1 thread, 91 MB on 8, 155 MB on 16, then 165–180 MB on 32 and 64. The file is never loaded whole: each thread reads its part in 64 KiB blocks. So the memory is what every running thread holds at the same moment: its current block, the Python objects parsed from it, its stack, and the memory its own allocator keeps (in the free-threaded build each thread allocates from its own heap), about 7–8

MB per thread. It stops growing after 16 because only 16 threads run at once; a thread that has finished its part releases its memory and the next one reuses it. The block size is what multiplies with the threads: with 4 MiB blocks, 16 threads needed 1.33 GB.

4. Rank the three versions of Task 1b by time. Which are correct? From fastest to slowest: v3 (0.22 s), v1 (8.05 s), v2 (24.0 s). v2 and v3 are correct, v1 is not: it counted only about 10 million of the 50 million numbers, a different number in each run.

- **v3** is fast and correct: each thread counts in its own local variables and nothing is shared until the end, when the main thread adds 16 triples. No thread ever waits for another. It is faster than Task 1a (0.53 s) because here the numbers are already parsed.
- **v1** loses updates. `counts[k] += 1` is three steps: read the value, add 1, write it back. If two threads read 100 at the same time, both write 101 and one increment is lost. With 16 threads on 3 counters, about 80% of the increments were lost. It is also slow: all 16 cores keep writing to the same list, so the cache line holding it travels from core to core on every write. The free-threaded interpreter keeps the list itself intact (it is never corrupted), but it does not make the read-add-write sequence atomic.
- **v2** is correct because the lock makes read-add-write one indivisible step, and the slowest because it takes the lock 50 million times, and 16 threads all want the same lock: at any moment one thread counts and fifteen wait. The work becomes sequential again, plus the cost of passing the lock from thread to thread.

With the GIL turned on, v1 gave the *correct* totals in 3 runs out of 3 (3.1 s each, `python3.14t -X gil=1 lab.py 1b data/numbers.txt 1`). Only one thread runs at a time and the interpreter switches threads only at certain points, so the race practically never shows. The code is still wrong, the GIL only hides the bug.

5. Compare Task 1c with Task 1a. Task 1a is faster: its best is 0.53 s, against 2.78 s for 1c, five times faster. In 1c one reader goes through the 50 million lines one by one and builds the batches, and this part is sequential: the workers can never be faster than the reader. In 1a every thread reads and splits its own part, so reading is parallel too. Task 1c stops getting faster already at 1 worker: 2.78 s with 1 worker, 2.78–2.97 s with 2 to 32, 3.33 s with 64. With 1 worker it is faster than 1a on 1 thread (2.78 s against 3.48 s) because it really uses two threads: the reader splits lines while the worker parses them, like a pipeline. More workers only wait on an empty queue and add memory (210 MB with 64).

6. What would happen with batches of a single line? And with a queue without a size limit? With batches of one line, every number would go through the queue, with a lock and a condition variable on every `put` and every `get`. I measured it on the first 5 million lines with 4 workers: 13.0 s with batches of 1 against 0.29 s with batches of 10 000, 45 times slower (about two minutes for the full file), and the peak memory rose to 500 MB. It is the same lesson as Task 1b v2: synchronising once per number costs more than the work itself.

With no size limit, the reader would never wait. If it is faster than the workers, the whole file piles up in the queue as Python objects, 2–3 GB for 50 million lines, so the memory would grow with the file instead of staying under 100 batches $\times$ 10 000 lines. On my machine the reader is the slower side, so an unlimited queue gave the same results (2.6–3.6 s, 46–82 MB): the queue never fills. The limit matters when the consumers are slower: it is back-pressure, it makes the producer wait instead of filling the memory.

7 Task 2 results

The 8 files are copied to the USB drive and read only from there. The task asks to unplug the drive before every run so that no data comes from the file cache. I reached the same state without unplugging it: before every run the drive was unmounted and mounted again with `udisksctl`, which removes its files from the page cache, and `fincore` confirmed that 0 bytes of the 8 files were cached when the run started. Had the files come from RAM, a run would have taken about 3.5 s, as in Task 1a, not 20 s. Each configuration ran 3 times and all the totals were correct.

Reading the 8 files once without parsing them (`cat` to `/dev/null`, after the same remount) took 19.5 s: the drive delivers about 19 MB/s.

threads	time (s)	speed-up	peak memory (MB)
1	21.853	1.00	125.7
2	21.406	1.02	225.9
4	20.378	1.07	421.8

Table 7.1 – Task 2, medians of 3 runs (`results/2.csv`)

The peak memory grows with the threads because every thread holds a whole 46 MB file in one buffer, as the task requires, and the buffer of its previous file is released only when the next `f.read()` returns. That is up to two files per thread, about 92 MB per thread on top of the 34 MB the program needs on one thread in Task 1a.

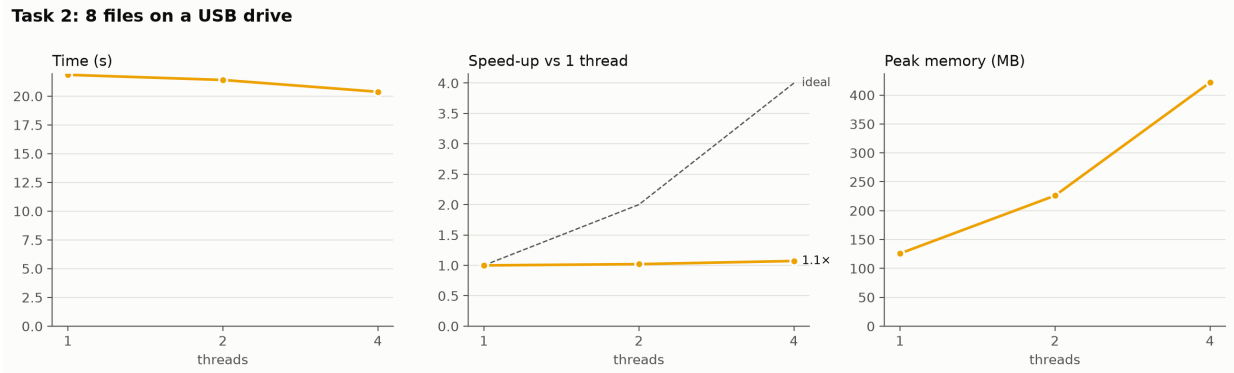


Figure 7.1 – Task 2: time, speed-up and peak memory against the number of threads

1. Compare the time on one thread with Task 1a on one thread. Is the work CPU-bound or I/O-bound on the USB drive? Task 1a on one thread parses the same 50 million numbers from the page cache in 3.48 s. Task 2 on one thread takes 21.85 s, 6.3 times longer, for the same parsing: the extra 18.4 s is waiting for the drive. Reading the files alone takes 19.5 s, about 89% of the run, while the CPU parses for only about 3.5 s of the 21.9 s. On the USB drive the work is **I/O-bound**: a faster drive would make it faster, more cores would not.

2. On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency? Hardly. 2 threads take 21.41 s (1.02×) and 4 threads 20.38 s (1.07×), while the peak memory grows from 126 to 422 MB. Concurrency helps I/O-bound work when the waits can overlap: a server that waits for many other machines is idle while it waits, so many requests can wait at the same time. Here every thread waits for the same device, and that device is already busy at its full speed of about 19 MB/s with one thread. More threads only put more requests in the queue of the same USB 2.0 drive; it does not read faster. The only thing the threads can overlap is the parsing (about 3.5 s) with the reading of other files, so the time cannot

drop below the 19.5 s the drive needs to deliver the data. 4 threads, at 20.4 s, are already close to that limit.

8 Conclusion

On my machine, counting the numbers of a file on the NVMe SSD is **CPU-bound**. Reading the whole 369 MB file without parsing it takes 48 ms (it is in the page cache), which is 1.4% of the 3.48 s that one thread needs to parse and count it; the rest is the interpreter turning text into integers. That is why the time drops almost in proportion to the threads up to the 8 physical cores (6.1×), and stops there: the limit is the number of cores, not the disk.

The experiments also show the two costs of sharing: shared counters without a lock are wrong (80% of the updates lost), and with a lock they are correct but 110 times slower than per-thread counters. The fastest correct design shares nothing while working and combines the results once at the end. Task 1c shows the other limit: a single producer is sequential, and no number of consumers can make the program faster than it.

Counting the same numbers from the USB drive is **I/O-bound**. The drive delivers about 19 MB/s, so reading the 369 MB takes 19.5 s while parsing them takes 3.5 s: one thread needs 21.9 s instead of 3.48 s, and 4 threads gain only 7% while the 8 cores are mostly idle.

I know which case is which from two measurements: the time of reading alone compared with the whole run, and whether more threads make the run faster. From the page cache, reading is 1.4% of the time and threads give 6.1×: CPU-bound. From the USB drive, reading is about 89% of the time and threads give 1.07×: I/O-bound.